Hospital Order Management System — Design Document

Course: CSCI-P532 | **Project:** 2 Week 1 | **Student:** Ishit Sutariya | **Stack:** Java 21 · Spring Boot 3.5.1 · Maven · Docker | **Architecture:** The Method 4-Layer

1 · LAYERED COMPONENT DIAGRAM

Client · Manager · Engine · Resource Access · Resource · Utility (cross-cutting — vertical right bar)

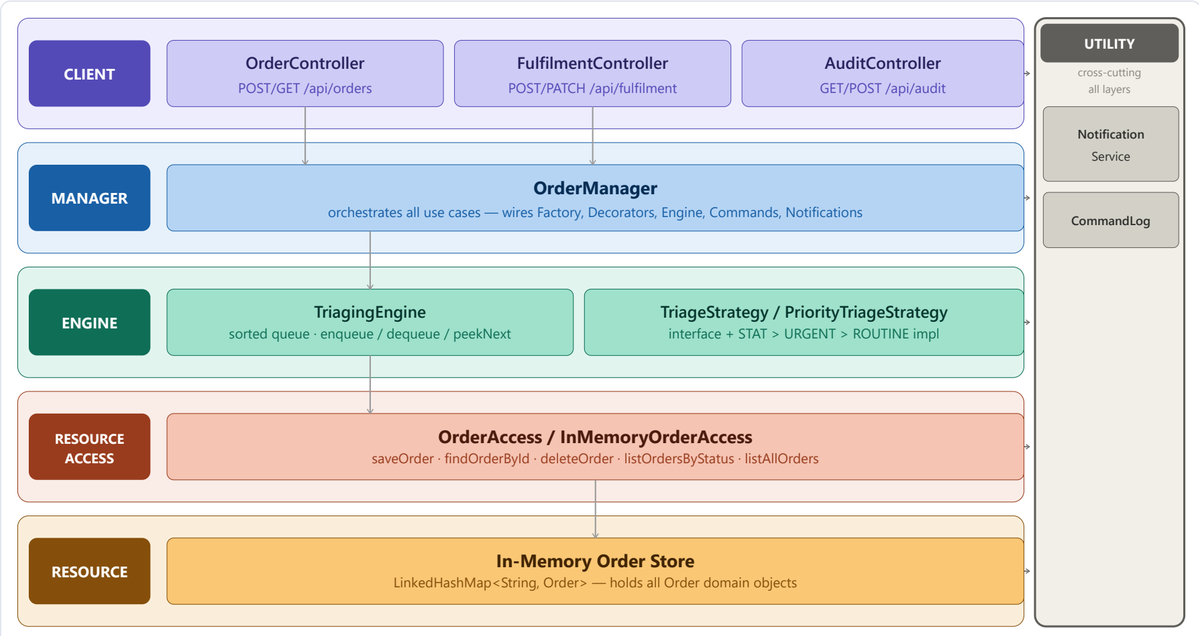

Figure 1 — Layered component diagram. Utility is a vertical bar on the right because it is cross-cutting — every layer reaches it independently (dashed arrows). Solid arrows show the strict top-down delegation path: Client → Manager → Engine → Resource Access → Resource. No layer may call upward.

2A · CALL CHAIN — USE CASE 1: SUBMIT AN ORDER

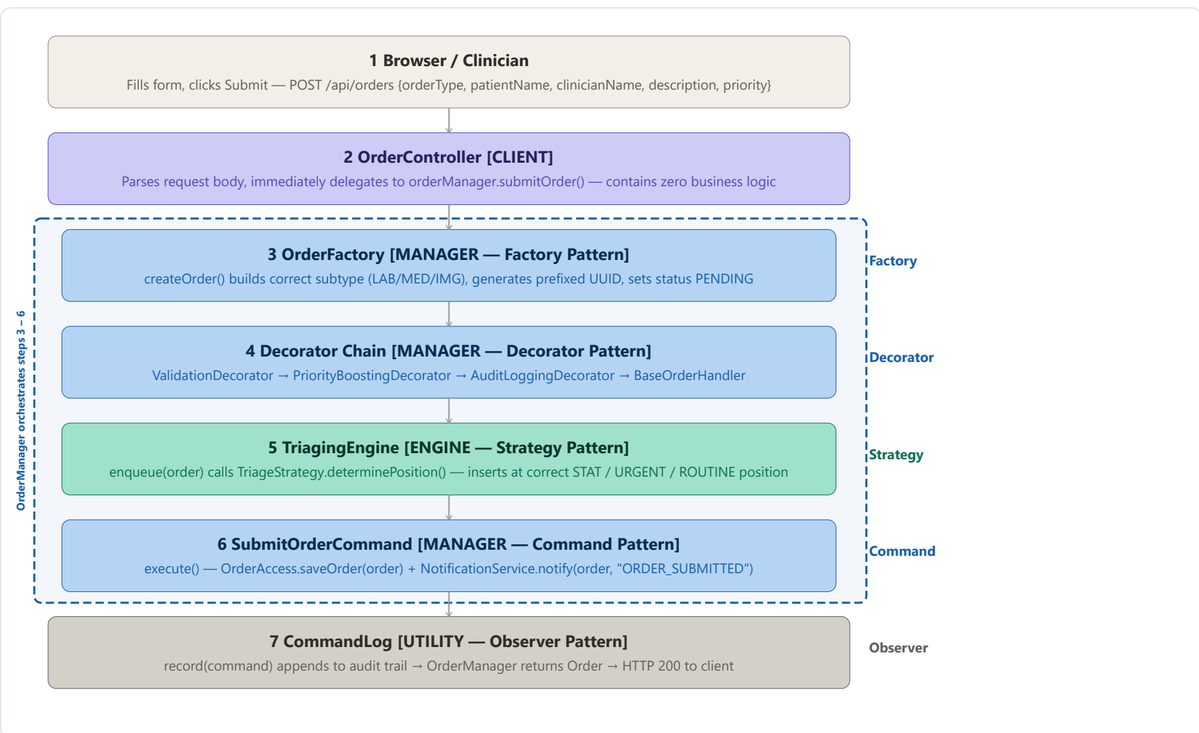

Figure 2 — Call chain for Use Case 1: Submit an Order. The dashed blue bracket shows OrderManager's orchestration scope; arrows show execution order, not direct caller-to-callee. OrderManager invokes steps 3–6 in sequence — TriagingEngine (step 5) returns its result to OrderManager, which then calls SubmitOrderCommand (step 6). Engine never calls Manager.

2B · CALL CHAIN — USE CASE 2: FULFIL AN ORDER (CLAIM + COMPLETE)

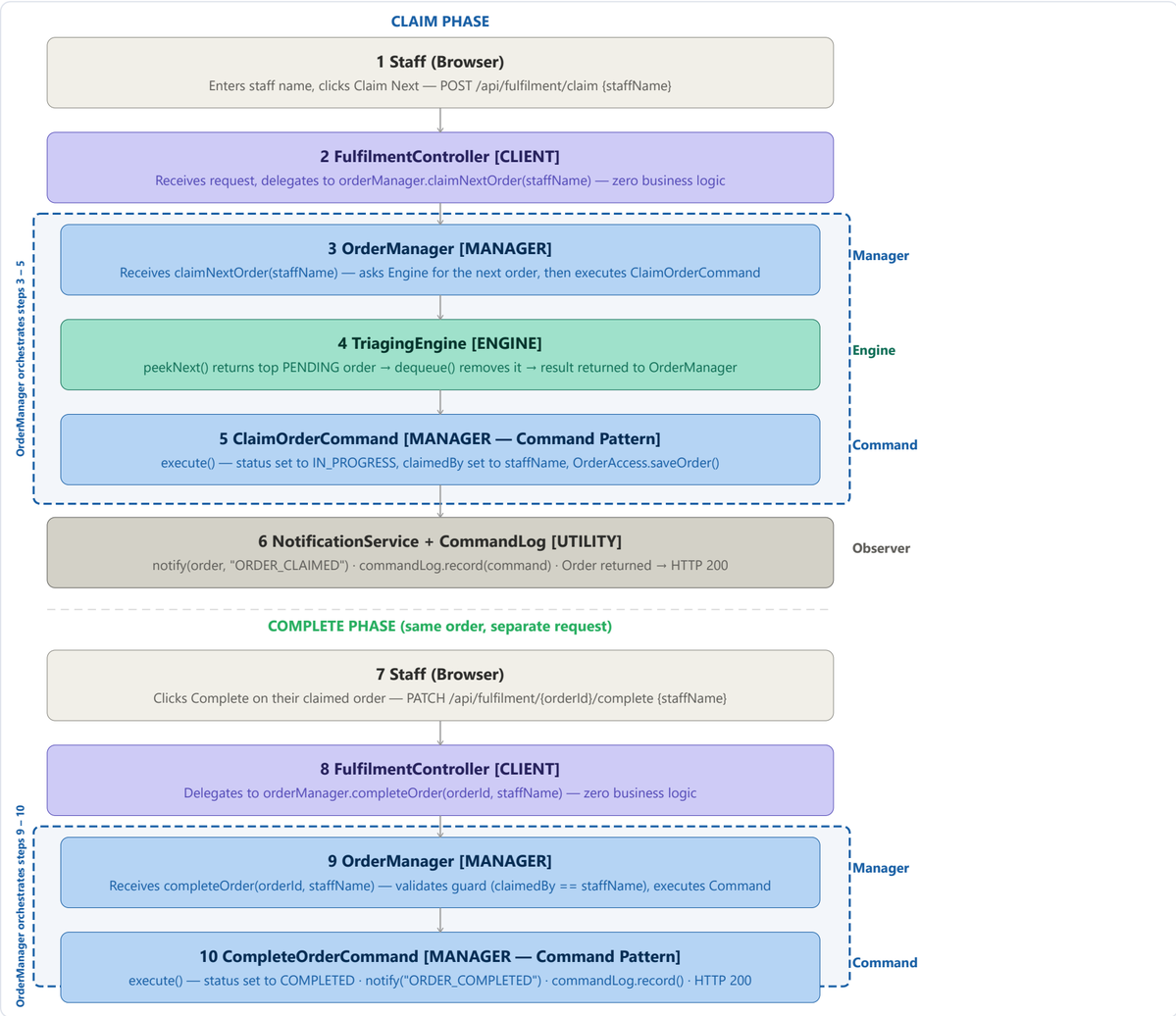

Figure 3 — Call chain for Use Case 2: Fulfil an Order. OrderManager is now an explicit step in both Claim and Complete phases. Client always delegates to Manager first — it never reaches Engine directly. Engine (step 4) returns its result to OrderManager, which then calls the Command — Engine does not call Manager. Claim (steps 1–6) and Complete (steps 7–10) are two separate HTTP requests.

3 · DESIGN PATTERN JUSTIFICATIONS

PATTERN	CLASS(ES)	LAYER	ONE-SENTENCE JUSTIFICATION
Strategy	`TriageStrategy` `PriorityTriageStrategy`	Engine	Encapsulates the triage algorithm behind an interface so the active sorting policy can be swapped at runtime by injecting a different implementation without modifying `TriagingEngine` or `OrderManager`.
Observer	`NotificationService` `ConsoleNotificationService`	Utility	Decouples order-status changes from the notification channel so switching from console logging to email or SMS requires only adding a new implementation — zero changes to any command or manager.
Decorator	`OrderHandler`, `ValidationDecorator`, `PriorityBoostingDecorator`, `AuditLoggingDecorator`, `BaseOrderHandler`	Manager	Stacks order-processing steps (validate, priority-boost, audit-log) transparently around a base handler so new steps can be added by creating a new decorator class without modifying any existing handler.
Factory	`OrderFactory`	Manager	Centralises creation of the correct Order subtype (LAB, MEDICATION, IMAGING) at a single point so all callers are decoupled from construction logic and adding a new type requires editing only one file.
Command	`Command`, `SubmitOrderCommand`, `ClaimOrderCommand`, `CompleteOrderCommand`, `CancelOrderCommand`, `CommandLog`	Manager	Encapsulates each order action as an object carrying everything needed to execute and undo it, enabling a complete audit trail and a ready-to-activate undo mechanism with no structural changes to `OrderManager`.